



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr



Chapitre 2 : Notion de la Programmation Orientée Objet

1. Notion de classe
2. Notion d'objets
3. Constructeur + Surcharge (Overloading)
4. L'héritage + Redéfinition (Overriding)
5. Polymorphisme
6. Classe Abstraite
7. Interfaces

I. Notion de classe

- Une classe est un **modèle** qui décrit la structure et le comportements des objets.
- Elle contient des attributs (variables) pour stocker les données et des méthodes pour définir le comportement.

Exemple d'une classe :

```
public class Voiture {
    String marque;
    String couleur;
    int vitesse;

    void accelerer() {
        vitesse = vitesse + 10;
    }
    void freiner() {
        vitesse = vitesse - 5;
    }
}
```

I. Notion de classe

Définition d'une classe :

```
modificateur class Nom_Classe {
    // instructions pour définir les champs et les méthodes de la classe
}
```

Un modificateur de **visibilité** (ou **d'accès**) indique si les autres classes de l'application pourront accéder ou non à la classe.

Les modificateurs d'accès d'une classe peut être :

- par défaut (aucun mot-clé) : La classe est accessible uniquement par les classes du même package.
- **public** : classe est accessible depuis n'importe quelle classe quel que soit le package.

Remarque : Un package est équivalent à un dossier, il sert à regrouper des classes.

1 - Définition des attributs :

Champs d'une classe qui reçoivent des données : ils sont tous visibles et accessibles par toutes les méthodes définies dans la classe.

Syntaxe :

modificateur type Nom_Attribut;

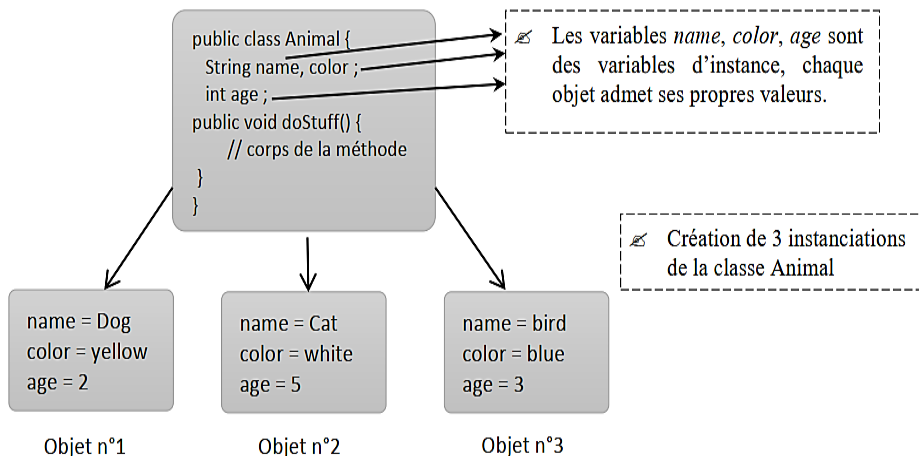
Un **modificateur d'accès** d'un attribut peut être :

- **public** : L'attribut est accessible depuis n'importe quelle autre classe.
- **protected** : L'attribut est accessible dans la même classe, dans les classes du même package et dans les classes héritées (même hors du package).
- par default (*aucun mot-clé*) : L'attribut est accessible uniquement dans le même package.
- **private** : L'attribut est accessible uniquement dans la classe où il est défini.

1 - Définition des attributs :

Dans une classe, on trouve trois types des variables :

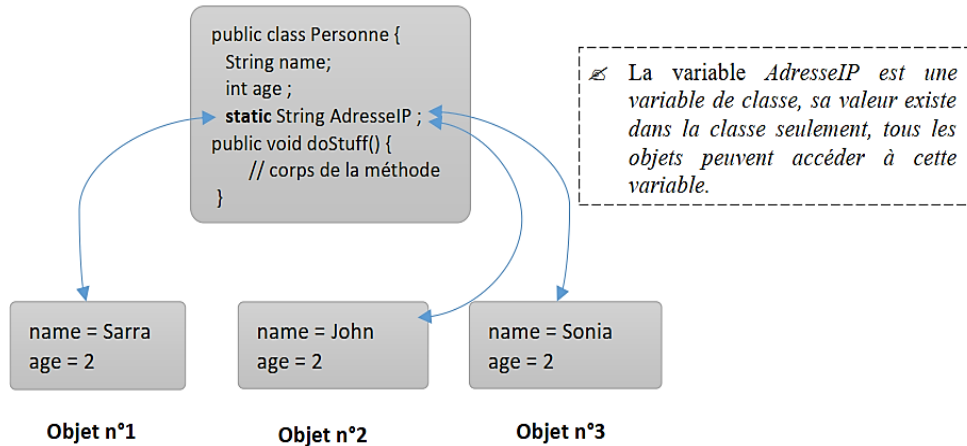
a. Variables d'instance : Sont des variables propres à chaque objet.



1 - Définition des attributs :

Dans une classe, on trouve trois types des variables :

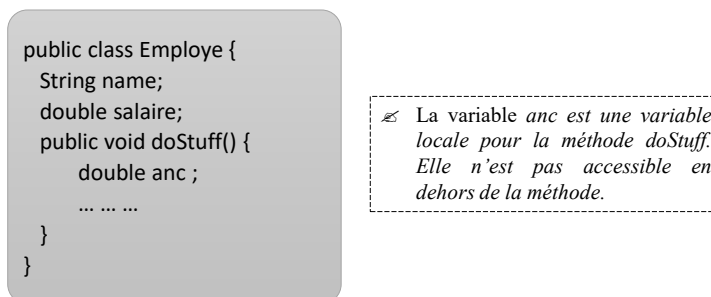
- b. Variables de classe** : Sont des variables qui appartiennent à la classe. Tous les objets partagent la même variable. Elles sont précédées par le mot clé **static**.



1 - Définition des attributs :

Dans une classe, on trouve trois types des variables :

- c. Variables locaux** : Ce sont des variables qui sont définies à l'intérieur d'une méthode. Leur visibilité est limitée au bloc de code dans lequel elles sont déclarées.



2 - Définition des méthodes :

La définition d'une méthode ressemble à celle d'une fonction dans les autres langages.

Syntaxe :

```

modificateur type nom_méthode ( type1 arg-1, type2 arg-2, ..., type-n arg-n )
{
    ..... Bloc d'instructions
}

```

Les **modificateurs d'accès** d'une méthode sont :

- **public** : La méthode est accessible depuis n'importe quelle autre classe.
- **protected** : La méthode est accessible dans sa propre classe, dans les classes du même package et dans les sous-classes (même hors du package).
- par défaut (aucun mot-clé) : Si aucun modificateur n'est spécifié, la méthode est accessible uniquement aux classes du même package.
- **private** : La méthode est accessible uniquement au sein de sa propre classe.

Exercice 1:

Donner la définition d'une classe nommée **Point** dont elle admet deux valeurs entières x et y (abscisse et ordonnée) et les méthodes suivantes :

- initialise() : pour initialiser les coordonnées d'un point à 0.
- deplace() : pour tradater les coordonnées d'un point de +2.
- affiche() : pour afficher les coordonnées d'un point.

Classe Point

Attributs
+ x: int (coordonnée entière)
+ y: int (coordonnée entière)
Méthodes
→ initialise() : void // pour initialiser les coordonnées d'un point à 0
→ deplace() : void // pour tradater les coordonnées d'un point de +2
→ affiche() : void // pour afficher les coordonnées d'un point

II. Notion d'objets

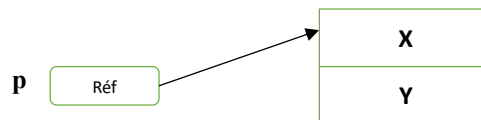
- Un objet est une instance d'une classe, pour chaque instance la structure est la même.
- Cependant, les valeurs des attributs peuvent varier d'un objet à l'autre,

Création d'un objet :

- La création d'un objet à partir d'une classe est appelée **instanciation**.
- Un objet est une **instance** de sa classe.
- Il est nécessaire de déclarer un objet avant de l'instancier.
- L'opérateur **new** permet de créer une instance de la classe et de l'associer à la variable objet.

Exemple de création d'un objet :

- Déclaration d'un objet **p** de type **Point**.
 $\Downarrow$ Point p;
- Création d'un objet p (**instanciation**) :
 $\Downarrow$ p = **new** Point();
- On peut faire la déclaration et l'instanciation ensemble :
 $\Downarrow$ Point p = new Point();
- La situation peut être schématisée dans la mémoire par :



- Le contenu du p est une référence de l'objet créé.

III. Constructeur

- Un constructeur est une méthode qui porte le **même nom de la classe** et **sans valeur de retour**.
- Le constructeur sera **appelé au moment de l'instanciation** de la classe pour créer un objet. Il est invoqué à l'aide de l'opérateur **new**.
- Il est nécessaire d'initialiser les attributs de la classe à instancier à l'aide du constructeur.
- Si une classe n'admet pas un constructeur, le **compilateur Java crée** automatiquement un **constructeur par défaut** et lui attribue comme nom, le nom de la classe et sans arguments.

Exercice 2 :

Créer une classe Java permettant de saisir deux points p et q et afficher l'un des messages suivant :

- « symétrie par rapport à l'axe de l'abscisse »
- « symétrie par rapport à l'axe de l'ordonnée »
- « symétrie par rapport au centre »
- « points quelconque ».

Le programme principal Java doit utiliser la classe Point déjà rencontrée.

i.e : Transformer la méthode initialise() de la classe Point en un constructeur, donner la définition complète de la classe Point.

Quelques règles sur le constructeur

1. Un constructeur ne fournit aucune valeur de retour, dans son en-tête aucun type ne doit figurer devant son nom, même le mot void.



```
class Voiture
{.....
    public void Voiture () // erreur de compilation
    {..... }
    .....
}
```

Quelques règles sur le constructeur

2. Dès qu'une classe possède au moins un constructeur, le constructeur pas défaut ne peut plus être utilisé.

```
class Point
{ .....
    public Point (int a, int b) // constructeur crée par le programmeur
    {.....
    }
}
```

```
//dans le programme principal
Point p = new Point (3,5) ; // OK
Point q = new Point () ; // erreur : on ne peut pas utiliser le constructeur
par défaut car il existe déjà un constructeur défini par le programmeur.
```


Quelques règles sur le constructeur

- Un constructeur ne peut pas être appelé directement comme une autre méthode (Le constructeur est appelé par le compilateur).

```
class Point
{ .....
    public Point (int a, int b) // constructeur crée par le programmeur
{.....
}

Point a = new Point (4,7) ;
// on peut appeler toutes les méthodes de la classe Point sauf le
constructeur.
a.affiche ( ) ; // Appel correct
a.Point (2,2) ; //Appel interdit.
```

Quelques règles sur le constructeur

- Dans une classe, on peut définir plusieurs constructeurs qui se différencient par leurs arguments.

```
class Point
{ .....
    public Point (int a, int b) // constructeur avec deux paramètres
    {
        x=a ; y=b ;
    }
    public Point (int a) // constructeur avec un seul paramètre
    {
        x=y=a
    }
} // on parle ici d'un constructeur surchargé

// Lors de l'instanciation, le programmeur à le choix d'utiliser entre le premier
ou le deuxième constructeur
Point a = new Point (1,2) ;
Point b = new Point (5) ;
```

Surcharge des méthodes (Overloading)

- La surcharge d'une méthode permet à une classe de définir plusieurs fois la même méthode mais avec des arguments différents (**signatures différentes**).
- Le compilateur choisi la méthode qui doit être appelée en fonction du nombre et du type des paramètres.
- Pour qu'une méthode soit considérée comme une surcharge valide, il doit y avoir une différence entre :
 - Le nombre de paramètres.
 - Le type des paramètres.
 - L'ordre des types des paramètres.

Initialisation par défaut des champs d'un objet

- Dès qu'un objet est instancié par l'opérateur **new** et avant l'appel du constructeur, les champs d'un objet sont initialisés automatiquement à une valeur par défaut bien définie.

Type du champ	Valeur par défaut
boolean	false
char	caractère de code nul
entier (byte, short, int, long)	0
flottant (float, double)	0.f ou 0.
Objet (Référence)	null

- Une variable locale n'est pas concernée par l'initialisation par défaut.
- Une variable d'instance peut être déclarée avec le mot **final**, dans ce cas il faut l'initialiser d'une façon explicite.

Autoréférence : le mot clé this

- Le mot-clé **this** en Java fait référence à l'instance courante de la classe.
- Utilisé pour accéder aux attributs de l'objet.
- « this » peut également être utilisé pour appeler un autre constructeur de la même classe.

Exemple 1 :

```
public class Voiture {
    private String couleur;
    private int annee;

    // Constructeur principal
    public Voiture(String couleur, int annee) {
        this.couleur = couleur;
        this.annee = annee;
    }

    // Constructeur secondaire
    public Voiture(String couleur) {
        this(couleur, 2025); // Appel au constructeur principal avec la valeur 2025 pour l'annee
    }
}
```

Autoréférence : le mot clé this

Exemple 2 :

```
class Point {
    private int x,y;

    public Point(int x, int y) {
        this.x = x ;
        this.y = y ;
        System.out.println("Constructeur deux arguments : " + x + " " + y);
    }

    public Point() {
        this(0,0);
        System.out.println("Constructeur sans arguments");
    }
}
```

```
public class OpeationThis {
    public static void main(String [] pm)
    {
        Point a = new Point(1,2);
        Point b = new Point();
    }
}
```

Appel du 1^{er} constructeur par à partir du 2^{ème} constructeur

exécution :

```
Constructeur deux arguments : 1 2
Constructeur deux arguments : 0 0
Constructeur sans arguments
```

Notion de package

- Un package en Java est un mécanisme qui permet de regrouper des classes, des interfaces et des sous-packages ensemble. Il permet de :
 - Organiser le code de manière logique, comme une sorte de dossier pour vos classes
 - Éviter les conflits de noms entre les classes
 - Contrôler l'accès aux classes et leurs membres grâce aux modificateurs d'accès.
- Pour créer un package, on utilise le mot-clé **package** en début de fichier.
- Pour utiliser une classe d'un autre package, il faut l'importer avec le mot-clé **import**.
- Exemple :
 - `import nom_package.nom_classe;` ➤ importe une seule classe (publique).
 - `import nom_package.*;` ➤ l'astérisque pour importer les classes publiques et n'importe pas les classes des sous-packages.

Exercice 3

- Soit la définition de la classe A :

```
class A {
    public void f (byte b)      { ..... }
    public void f (int n)      { ..... }
    public void f (int n, int m) { ..... }
    public void f (float x)     { ..... }
    public void f (int n, double y) { ..... }
    public void f (double y)    { ..... }
    public void f (long q, float x) { ..... }
}
```

Question :

Quelles sont les instructions correctes et, dans ce cas, quelles sont les méthodes appelées et les éventuelles conversions mises en jeu ?

1°) <code>a.f(b);</code>	6°) <code>a.f(b++);</code>
2°) <code>a.f(n, q);</code>	7°) <code>a.f(p, n);</code>
3°) <code>a.f(p);</code>	8°) <code>a.f(b, x);</code>
4°) <code>a.f(q);</code>	9°) <code>a.f(n, y);</code>
5°) <code>a.f(b+1);</code>	10°) <code>a.f(p,q);</code>

Avec les déclarations suivantes :

`A a ; byte b ; short p ; int n ; long q ; float x ; double y ;`

Exercice 4

- Que fournit le programme Java suivant ?

```
class Inconnue {
    public Inconnue (int coeff){
        nbre *= coeff ;
        nbre += decal ;
    }
    public void affiche (){
        System.out.println ("nbre = " + nbre + " decal = " + decal) ;
    }
    private int nbre = 20 ;
    private int decal ;
}
public class Exercice4 {
    public static void main (String args[]){
        Inconnue a = new Inconnue (5) ;
        a.affiche() ;
    }
}
```

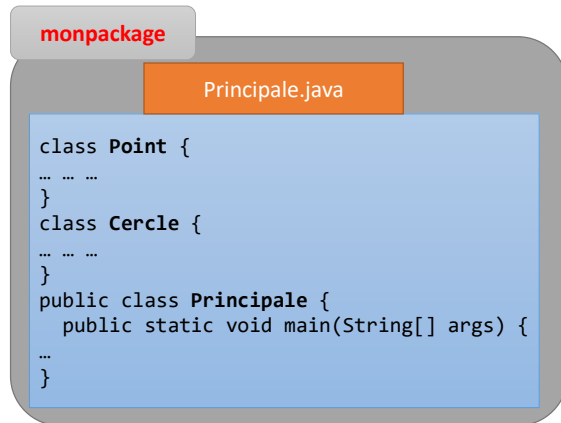
Exercice 5

1. Définir la classe Point composé par deux champs et un constructeur qui initialise les attributs passés en paramètres et la méthode distance qui retourne la distance de ce point par rapport au centre.
2. Définir la classe Cercle composé par un nom et un rayon et un constructeur qui permet d'initialiser les attributs explicitement.
3. Définir la classe Principale qui permet de tester si un point appartient au cercle concentrique ou non.

Exercice 5 (Conception des classes)

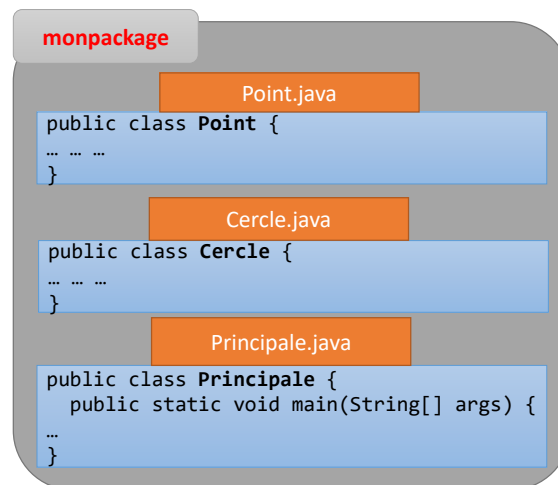
→ Trois classes : **Point**, **Cercle**, **Principale**. Comment concevoir notre projet ?

1^{ère} méthode : Un seul package contenant un seul fichier source (.java) qui porte le même nom que la classe contenant la méthode main()



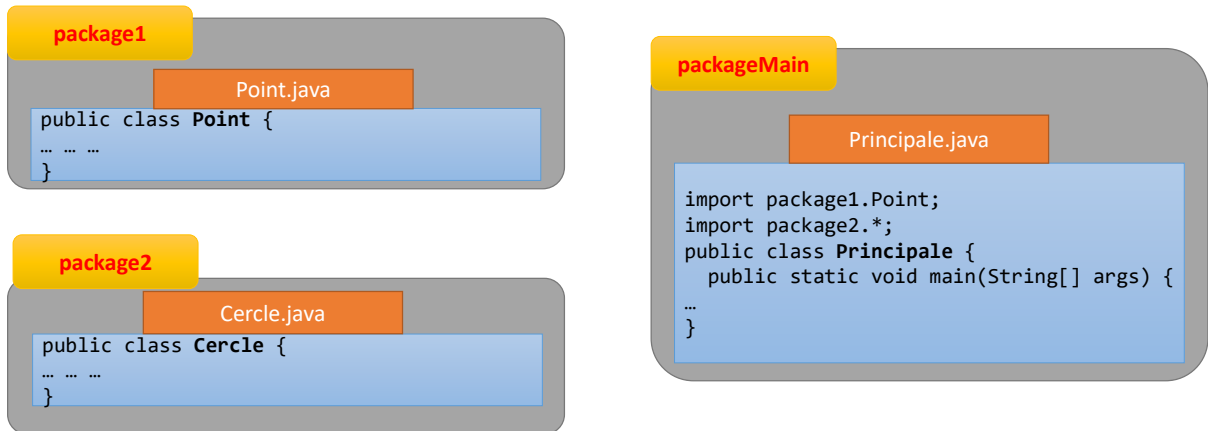
Exercice 5 (Conception des classes)

2^{ème} méthode : Un seul package contenant un seul fichier source (.java) qui porte le même nom que la classe contenant la méthode main()



Exercice 5 (Conception des classes)

3^{ème} méthode : Trois package, chacun doit contenir un seul fichier source (.java). Si une classe à besoin d'utiliser une autre classe, elle doit importer son package, par le mot clé import



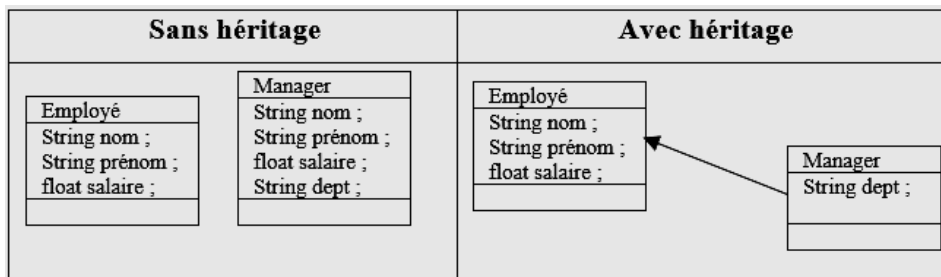
Exercice 6

On veut créer une classe **Livre** ayant pour attribut obligatoire le nom de l'auteur, le titre et comme attribut optionnel le nombre de pages. Les attributs seront privés, et on utilisera des méthodes d'accès publiques (getters et setters).

1. Donner la définition des constructeurs nécessaires
2. Définir complètement la classe Livre.
3. On veut maintenant connaître le nombre total de livres instanciés « **nbLivre** ». Proposez une solution à ce problème. Modifiez les constructeurs de manière à ce que cela fonctionne.
4. Ajouter le champ « **numLivre** » à chaque livre qui contiendra le numéro du livre dans l'ordre de création des objets (ordre d'instanciation)

IV. Héritage

- Supposons que nous avons une classe intitulée « **Employé** » contient les champs privés suivants : *nom*, *prénom*, *salaire*.
- Créer une autre classe intitulée « **Manager** ». Un manager a les mêmes attributs que la classe Employé (*nom*, *prénom*, *salaire*) de plus il possède le champ *département*.
- La relation **d'héritage** est applicable entre la classe « Employé » et la classe « Manager », car ils sont reliés par la relation **Is-A** (est un) : Un Manager **est un** Employé.



- L'héritage est un mécanisme qui permet à une classe d'hériter des propriétés et méthodes d'une autre classe. Cela favorise la réutilisation du code et permet de créer une hiérarchie de classes.
- La définition de deux classes « **Employe** » et « **Manager** » avec la relation d'héritage

```
class Employe {
    String nom;
    String prenom;
    double salaire;
    void fe() {...}
}

class Manager extends Employe {
    String dept ;
}
```

- La classe Employe est désignée par : classe de base, classe parent ou super classe.
- La classe Manager est désignée par : classe dérivée, classe fille ou sous classe.

Règles sur le concept d'héritage :

- Le mot clé **extends** permet d'utiliser la technique d'héritage.
- Java ne supporte que l'héritage simple (une classe ne peut hériter que d'une seule classe). Pas d'héritage multiple.
- Les membres (attributs & méthodes) **non privés** hérités de la classe de base sont accessibles dans la classe dérivée à l'aide du mot clé **this**.
- Les attributs déclarés **private** restent inaccessibles directement pour la classe dérivée, c'est l'objectif de **l'encapsulation** : contrôler l'accès aux attributs.
- Pour pouvoir accéder aux attributs privés, il faut utiliser les méthodes d'accès de la classe de base (**getters et setters**).

Règles sur le concept d'héritage :

- La classe dérivée peut redéfinir (**overriding**) ou surcharger les méthodes héritées.
- La redéfinition (overriding) : changer le corps de la méthode héritée et garder la même signature.
- Toute classe Java hérite implicitement de la classe **Object** (java.lang.Object).
- Quelques méthodes de la classe java.lang.Object :
 - public boolean **equals**(Object obj) : Sert à comparer deux objets pour vérifier s'ils sont « égaux ».
 - public String **toString**() : Sert à obtenir une représentation textuelle d'un objet. (nom_package.nom_classe@code).

Construction et initialisation d'un objet dérivé :

- Pour construire un objet de la classe dérivée, il est obligatoire d'appeler le constructeur de la classe de base.
- L'appel du constructeur de la classe de base se fait avec le mot clé **super()**, qui doit être placé sur la 1^{ère} ligne du constructeur de la classe dérivée.
- La fonction `super()` : permet d'initialiser les attributs privés de la classe de base via le constructeur parent et d'appeler les méthodes de la classe de base en cas de redéfinition.

Exercice :

- Donner le code Java complet de deux classes « Employe » et « Manager »

```
package Heritage;

public class Employe {
    private String nom;
    private String prenom;
    private double salaire;
    public Employe(String nom, String prenom, double salaire){
        System.out.println("Constructeur Employe");
        this.nom = nom;
        this.prenom = prenom;
        this.salaire = salaire;
    }
    public void affiche() {
        System.out.println("Nom: "+nom+" || Prenom: "+prenom+" || Salaire: "+salaire);
    }
}
```

```
package Heritage;
public class Manager extends Employe{
    private String dep;
    public Manager(String nom, String prenom, String dep, double salaire) {
        super(nom, prenom, salaire);
        System.out.println("Constructeur Manager");
        this.dep = dep;
    }
    public void affiche() {
        super.affiche();
        System.out.println("Departement: "+dep);
    }
}
```

```
package Heritage;
public class Principale {
    public static void main(String[] args) {
        Manager m = new Manager("Bob", "Alice", "Security", 3500);
        m.affiche();
    }
}
```

Cas particuliers d'appel de constructeur :

- Si la classe dérivée B et la classe de base A n'admettent pas de constructeur, alors la création d'un objet B entraîne l'appel du constructeur par défaut de B qui appelle lui-même le constructeur par défaut de A.

```
class A {
    ..... // pas de constructeur dans la classe A
}
class B extends A {
    ..... // pas de constructeur dans la classe B
}

B b = new B() ; // Construction d'un objet → appel au constructeur par
                défaut de B → appel au constructeur par défaut de A
```

Cas particuliers d'appel de constructeur :

- Comment se fait la création d'un objet de la classe B dans ce cas ?

```
class A {
    public A() {.....} // constructeur 1 (par défaut)
    public A(int n) {.....} // constructeur 2 (explicite)
}
class B extends A {
    ..... // pas de constructeur dans la classe B
}
B b = new B() ;
```

La construction d'un objet de B entraîne :

- Appel au constructeur par défaut de la classe B
- Appel au constructeur **1** de la classe A

Cas particuliers d'appel de constructeur :

- Comment se fait la création d'un objet de la classe B dans ce cas ?

```
class A {
    public A(int a) {...} // constructeur explicite
}
class B extends A {
    ..... // constructeur par défaut
}
B b = new B() ; → Tentative d'instanciation de la classe B
```

- Dans ce cas, on obtient une **erreur de compilation**, car le constructeur par défaut de B cherche à appeler un constructeur sans argument de A.

Redéfinition des méthodes (Overriding)

```
class Employe {
    private String prenom ;
    private String nom ;
    private double salaire ;
    public void affiche() {
        System.out.println("Bonjour Mr : " + nom + " " + prenom);
    }
}
class Manager extends Employe {
    private String dept ;
}
Employe e = new Employe() ;
Manager m = new Manager();
e.affiche();
m.affiche();
```

Quel sera le résultat de l'appel de la méthode affiche pour chaque objet ?

- **e.affiche()** : affiche le nom et le prénom relative à l'objet e.
- **m.affiche()** : affiche le nom et le prénom d'un objet m de type Manager (méthode affiche héritée), mais il n y a pas un moyen d'afficher le nom de département !

Redéfinition des méthodes (Overriding)

- Pour résoudre ce problème, nous allons définir dans la classe Manager une méthode portant le même nom que celle héritée et qui permet d'afficher les informations privées de la classe dérivée ainsi que le nom de département.
- On parle alors d'une **redéfinition d'une méthode** de la classe de base : elle permet à une classe dérivée de redéfinir une méthode de sa classe de base avec une nouvelle implémentation.
- La définition d'une méthode redéfinit masque la méthode dans la classe de base.

Redéfinition des méthodes (Overriding)

```
class Employe {
    private String nom, prenom ;
    public void affiche() {
        System.out.println("Bonjour Mr : " + nom + prenom);
    }
}
class Manager extends Employe {
    String dept ;
    public void affiche () {
        super.affiche(); //appel de la méthode affiche de la super-classe
        System.out.println("Votre département est : " + dept);
    }
}
```

Les champs **nom** et **prenom** sont privés, et la classe Employe ne dispose pas des méthodes **getters** pour accéder à ces champs privés, par conséquent nous allons redéfinir la méthode **affiche** de la classe de base.

Remarques :

- Une **classe** qui est définie par le mot clé **final** ne peut pas être héritée.
- Une **méthode** déclarée par le mot clé **final** ne peut pas être redéfinie dans une classe dérivée.
- Lors de la redéfinition d'une méthode, il faut respecter sa signature ainsi que sa valeur de retour.

Exercice 7

On dispose la classe Point suivante :

```
class Point {
    private int x, y ;
    public Point (int x, int y) {
        this.x = x ; this.y = y ;
    }
    public void affiche () {
        System.out.println ("Coordonnees : " + x + " --- " + y) ;
    }
}
```

Questions :

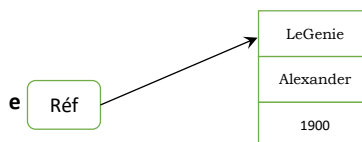
1°) Implémenter la classe **PointPlus**, dérivée de la classe Point, permettant de manipuler des points définis par leurs coordonnées (entières) et un nom (caractère). On y implémentera les méthodes suivantes :

- **constructeur** : pour définir les coordonnées et le nom d'un objet de type PointPlus.
- **affiche** : pour afficher les coordonnées et le nom d'un objet de type PointPlus.

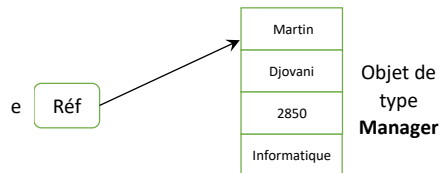
2°) Écrire un programme principal **TestPointPlus** utilisant la classe PointPlus.

V. Polymorphisme

- Le polymorphisme en Java est la capacité d'un objet à prendre plusieurs formes.
- Poly = plusieurs et Morphisme = forme
- Le polymorphisme est une conséquence directe de l'héritage.
- Soit la déclaration et la définition d'un objet de type Employe :
 - Employe e = new Employe ("LeGenie" , "Alexander" , 2700);



- Si on utilise la technique de polymorphisme, Java autorise ce type d'affectation :
 - **Employe** e = new **Manager** ("Martin", "Djovani", 2850 , "Informatique");



- L'objet « **e** » est de type « **Employe** » mais il prend la forme de l'objet « **Manager** ».
- ↳ Java permet d'affecter à une variable la référence vers un objet d'un type dérivé.
- e.affiche() : invoque la méthode affiche() de la classe Manager.
- La méthode affiche() s'adapte au comportement de l'objet référencé, c'est le grand avantage du polymorphisme.
- L'opérateur **instanceof** permet de vérifier si un objet est une instance d'une classe spécifique ou implémente une interface donnée.

Exercice 8

- Créer la classe de base « **Employe** » contient les champs privés suivants : nom, prénom, salaire.
- Créer la classe « **Manager** » qui possède les mêmes attributs que la classe Employe de plus elle possède le champ *département*.
- Créer la classe « **Adviser** » qui a les mêmes attributs que la classe Employe, de plus elle possède le champ *specialite*.
- L'accès aux champs privés de la classe de base se font par des **getters**.
- Dans la classe principale (main) :
 - Réaliser l'instanciation de deux Manager et deux Adviser.
 - Est-il possible de regrouper les 4 objets dans un tableau ? si oui proposer votre solution.

VI. Classe abstraite

- Une classe abstraite est une classe qui ne peut pas être instanciée. Elle peut servir que de classe de base pour une dérivation.
- Une classe abstraite est définie par le mot-clé **abstract**.
- Dans une classe abstraite, on peut trouver :
 - Des champs qui seront hérités.
 - Des méthodes concrètes.
 - Des méthodes abstraites : méthodes qui ne sont pas définies, c'est-à-dire une méthode qui n'admet que sa signature et son type de retour.

▪ Syntaxe :

```
abstract class A {
    private int champ;
    public void myF () {
        //corps de la méthode myF
    }
    public abstract void myG(int x); // méthode abstraite, pas d'implémentation.
}
```

- Le recours aux classes abstraites facilite largement la conception orientée objet.
- On peut placer dans une classe abstraite toutes les fonctionnalités dont on souhaite disposer pour les classes dérivées.

Règles avec les classes abstraites

- Une classe abstraite, appelée aussi classe non concrète, est une classe **non instanciable**.
- La redéfinition d'une méthode abstraite doit respecter sa signature et son type de retour.
- Une méthode abstraite doit être accessible pour que la classe dérivée puisse la définir.
- Une classe dérivée d'une classe abstraite n'est pas obligée de redéfinir toutes les méthodes abstraites de sa classe de base, dans ce cas elle reste aussi une classe abstraite.

```
abstract class B extends A {
    public abstract void myG(int x); // Ok
}
```

Règles avec les classes abstraites

- Une méthode **static** est une méthode qui appartient à la classe elle-même et non à une instance de la classe.
- On peut appeler une méthode **static** par le nom de la classe (pas besoin d'instance).
Exemple : Math.sqrt(...);
- Une méthode **static** ne peut pas être abstraite, car une méthode statique n'est pas destinée à être redéfinie.
- Une méthode **final** ne peut pas être abstract, car une méthode final ne peut pas être redéfinie (*best implementation*).
- Une classe abstraite admet un constructeur qui sera invoqué si un objet d'une classe dérivée sera créée.

Exercice 9

- Créer une classe abstraite nommée « **Animal** » contenant un champ privé « nom », un champ statique « nbre » permettant de calculer le nombre total des objets créés et un constructeur qui permet d'initialiser le nom et la méthode abstraite « parler ».
- Ajouter deux méthodes get/set pour accéder au champ privé « nom ».
- Créer deux classes intitulées « **Chien** » et « **Chat** » dérivées de la classe « Animal » implémentant chacune la méthode parler.
- Créer la classe principale « **TestAnimal** » permettant d'instancier deux objets l'un de type Chat et l'autre de type Chien et invoquer la méthode parler. Créer un tableau de type Animal permettant d'invoquer la méthode parler de toutes les objets créés. Déterminez le nombre de chaque objet créé.

VII. Interfaces

- Une interface est un contrat que les classes doivent respecter !
- Une interface est une structure qui permet de spécifier un ensemble des méthodes abstraites que les classes qui implémentent l'interface doivent obligatoirement fournir son implémentation.
- Dans une interface on peut trouver des constantes, qui sont déclarées comme *public*, *static* et *final* et qui doivent être initialisées.
- Depuis Java 8, une interface peut contenir des méthodes par défaut (**default**) avec une implémentation, (implémentation par défaut).
- Dans une interface, toutes les méthodes sont implicitement **public** & **abstract**. Tous les attributs sont implicitement **public**, **static** et **final**.

```
public interface MyI {
    int c = 19 ;
    void f(int n) ;
    default void g(){...} ;
}
```

VII. Interfaces

- Pour préciser qu'une classe implémente une interface on utilise le mot clé **implements** comme suit :

```
class A implements MyI {
    // A doit définir la méthode f de l'interface MyI
    void f(int x) {...}
}
```

- Une classe peut implémenter plusieurs interface.

```
class A implements I1, I2 {
    // Dans ce cas, A doit définir toutes les méthodes
    // dans l'interface I1 et I2.
}
```

- La définition des méthodes des interfaces implémentées par la classe doit être public.
- Une interface peut hériter une ou plusieurs interfaces

```
interface Myk extends MyI, MyJ {
}
```

Interface et polymorphisme

```
class SmartPhone extends Phone implements ICamera, IMusicPlayer
{
    void videoCall() { ... ... }
    void showVideo() { ... ... }
    void recordVideo() { ... ... }
    void playMusic() { ... ... }
    void stopMusic() { ... ... }
}
```

```
class Phone
{
    void call() { ... ... }
    void sms() { ... ... }
}
```

```
interface ICamera
{
    void showVideo ();
    void recordVideo ();
}
```

```
interface IMusicPlayer
{
    void playMusic();
    void stopMusic();
}
```

Les quelles de ces instructions sont valide ?

- SmartPhone s = new SmartPhone();
- Phone p = new SmartPhone();
- ICamera ic = new SmartPhone();
- ICamera ip = new Phone();
- IMusicPlayer imp = new SmartPhone();
- ICamera ic = s;
- IMusicPlayer imp = s;

Interface et polymorphisme

```
class SmartPhone extends Phone implements ICamera, IMusicPlayer
{
    void videoCall() { ... }
    void showVideo() { ... }
    void recordVideo() { ... }
    void playMusic() { ... }
    void stopMusic() { ... }
}
```

```
class Phone
{
    void call() { ... }
    void sms() { ... }
}
```

```
interface ICamera
{
    void showVideo();
    void recordVideo();
}
```

```
interface IMusicPlayer
{
    void playMusic();
    void stopMusic();
}
```

Combien de méthodes peut-on invoquer pour chaque objet et lesquelles ?

- Smartphone **s** = new SmartPhone();
- Phone **p** = new SmartPhone();
- ICamera **ic** = new SmartPhone();
- IMusicPlayer **imp** = new SmartPhone();

Interface et polymorphisme

- On peut définir des variables de type interface

```
public interface I { ... }
I i;
```

- Si A implémente l'interface I, alors on peut créer une référence i qui pointe vers un objet de A.

```
class A implements I { ... }
I i = new A(); // OK
```

- A travers la variable i, on peut appeler les méthodes redéfinies dans la classe A.
- ↳ L'appel **i.f();** est autorisé.

- Une référence d'interface peut pointer sur n'importe quel objet d'une classe qui implémente cette interface.

i

Réf

```
void f(...) {
    ...
}
```

- Objet de la classe A
- A implémente l'interface I

Exercice 10

- On vous demande d'écrire l'interface **Affichable** contenant la méthode `affiche()`.
- Ecrire la classe **Entier** qui admet un champ privé de type entier, qui sera initialisé par le constructeur et implémente l'interface **Affichable**.
- Ecrire la classe **Flottant** qui admet un champ privé de type float, qui sera initialisé par le constructeur et implémente l'interface **Affichable**.
- Dans la classe principale, créer un tableau de références **Affichable** pouvant contenir des objets de type **Entier** et **Flottant**.
- Remplir ce tableau par des objets **Entier** et des objets **Flottant**.
- Parcourir le tableau et invoquer la méthode `affiche()` de chaque objet.
- Utiliser l'opérateur *instanceof* pour calculer le nombre des objets **Entier** et **Flottant**.